Intreruperi

Privire de ansamblu

- De fiecare dată când este declanșată o întrerupere, indexul este incrementat și ora curentă este salvată.
- Diferența de timp este calculată și salvată în matrice (cu excepția primei declanșări a întreruperii, când indicele este 0).
- Când a avut loc numărul maxim de intrări, blocul interior în buclă rulează și imprimă toate valorile pe portul serial.

```
const int irReceiverPin = 2;             //pin the receiver is connected to
const int numberOfEntries = 64;

volatile unsigned long microseconds;
volatile byte index = 0;
volatile unsigned long results[numberOfEntries];

void setup()
{
  pinMode(irReceiverPin, INPUT);
  Serial.begin(9600);
  attachInterrupt(0, analyze, CHANGE);  // encoder pin on interrupt 0 (pin 2);
  results[0]=0;
}

void loop()
{
  if(index >= numberOfEntries)
  {
    Serial.println("Durations in Microseconds are:") ;
    for( byte i=0; i < numberOfEntries; i++)
    {
      Serial.println(results[i]);
    }
    while(1)
      ;
  }
  delay(1000);
}

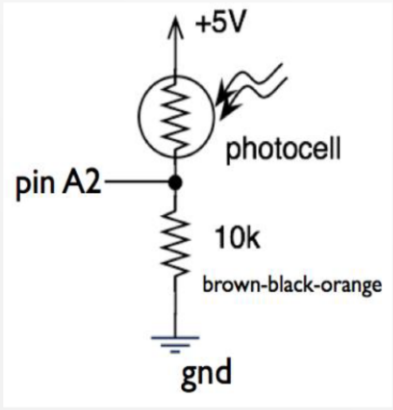
void analyze()
{
  if(index < numberOfEntries  )
  {
    if(index > 0)
    {
      results[index] = micros() - microseconds;
    }
    index = index + 1;
  }
  microseconds = micros();
}
```

Lista întreruperi - prioritati

Vector no.	Program address	Source	Interrupt definition
1	0000	RESET	External Pin, Power-on Reset, Brown-out Reset, Watchdog Reset and JTAG AWR Reset
2	0002	INT0	External Interrupt Request 0
3	0004	INT1	External Interrupt Request 1
4	0006	PCINT0	Pin Change Interrupt Request 0
5	0008	PCINT1	Pin Change Interrupt Request 1
6	000A	PCINT2	Pin Change Interrupt Request 2
7	000C	WDT	Watchdog Time-out Interrupt
8	000E	TIMER2_COMPA	Timer/Counter2 Compare Match A
9	0010	TIMER2_COMPB	Timer/Counter2 Compare Match B
10	0012	TIMER2_OVF	Timer/Counter 2 Overflow
11	0014	TIMER1_CAPT	Timer/Counter1 Capture Event
12	0016	TIMER1_COMPA	Timer/Counter1 Compare Match A
13	0018	TIMER1_COMPB	Timer/Counter1 Compare Match B
14	001A	TIMER1_OVF	Timer/Counter1 Overflow
15	001C	TIMER0_COMPA	Timer/Counter0 Compare Match A
16	001E	TIMER0_COMPB	Timer/Counter0 Compare Match B
17	0020	TIMER0_OVF	Timer/Counter0 Overflow
18	0022	SPI_STC	SPI Serial Transfer Complete
19	0024	USART0_RX	USART0 Rx Complete
20	0026	USART0_UDRE	USART0 Data Register Empty
21	0028	USART0_TX	USART0 Tx Complete
22	002A	ADC	ADC Conversion Complete
23	002C	EE_READY	EEPROM Ready
24	002E	ANALOG_COMP	Analog Comparator
25	0030	TWI	Two-Wire Serial Interface
26	0032	SPM_READY	Store Program Memory Ready

Ce face următorul cod?

• Dar ce culoare/valoare are rezistenta din schemă?




```
int pin = 13;
volatile int state = LOW;

void setup()
{
  pinMode(pin, OUTPUT);
  attachInterrupt(0, blink, CHANGE);
}

void loop()
{
  digitalWrite(pin, state);
}

void blink()
{
  state = !state;
}
```

The code you provided is an Arduino sketch that sets up an interrupt-based blinking functionality on pin 13. Here's what the code does:

1. Variable and Pin Declaration:

- `int pin = 13;`: Declares an integer variable `pin` and assigns it the value 13. This represents the pin number on the Arduino board.

- `volatile int state = LOW;`: Declares a volatile integer variable `state` and initializes it with the value `LOW`. The `volatile` keyword is used to indicate that the variable can be modified by an interrupt.

2. `setup()` Function:

- `pinMode(pin, OUTPUT);`: Sets the specified pin (`pin 13`) as an output pin.

- `attachInterrupt(0, blink, CHANGE);`: Attaches an interrupt to interrupt pin 0 (which corresponds to digital pin 2 on most Arduino boards). The interrupt is triggered on any change of the signal on that pin, and it will call the `blink()` function.

3. `loop()` Function:

- `digitalWrite(pin, state);`: Writes the value of the `state` variable to the specified pin (`pin 13`). This will turn the LED connected to that pin on or off, depending on the state.

4. `blink()` Function:

- This function is called whenever there is a change on interrupt pin 0 (digital pin 2).

- `state = !state;`: Inverts the value of the `state` variable using the logical NOT operator (`!`). So if `state` is currently `LOW`, it will be set to `HIGH`, and vice versa.

Overall, this code sets up an interrupt-based blinking functionality for an LED connected to pin 13. Whenever there is a change on interrupt pin 0 (digital pin 2), the `blink()` function is called, which toggles the `state` variable between `HIGH` and `LOW`. The `loop()` function continuously updates the LED state based on the value of `state`, resulting in the LED blinking on and off.

Subiecte PM

De la Rosner 2023:

(ex1) [3p] Ce fel de ADC este implementat pe ATmega 328P?
Care este frecventa maxima a unui semnal ce poate sa fie esantionat corect folosind acest ADC intern?
(ex2) [3p] Comparatie intre I2C si SPI.
(ex3) Se dă un senzor cu următorii parametri:
Tensiune de alimentare 5V; Consum senzor: 1uA în sleep/ 10mA în active mode; Domeniu de ieșire: 1 - 4V; Rata maximă de eșantionare de interes: o dată pe minut; Eficiența sursei de alimantare se consideră 80%.
Să se propună:
-[2p] O schemă bloc centrată în jurului unui uC 328P, cât mai eficientă din punct de vedere consum;
-[4p] Metode software de maximizare a autonomiei în timp/ reducere a consumului;

1) ADC cu aproximari succesive. Cu prescaler de 2, rezulta 16MHz/2/2 = 4MHz frecventa maxima
2) SPI e pe 4 fire, I2C pe 2 fire; SPI e full-duplex, I2C e half-duplex. SPI are un singur master, I2C poate sa aibe mai multi; Si SPI si I2C pot sa aibe mai multi slaves
3) limitezi amperajul si la 3b si eu cred a are treaba cu timere si intreruperi

[Sursa de Alimentare] --> [Regulator de Tensiune] --> [Unitate de Gestionare a Energiei] --> [Microcontroller 328P]

|

|

v

[Senzor] --> [Memorie de Stocare]

Sursa de Alimentare: Este sursa de tensiune (de exemplu, baterie sau adaptor de alimentare) care furnizează tensiunea necesară sistemului.

Regulator de Tensiune: Aceasta este o etapă de reglare a tensiunii care convertește tensiunea de la sursa de alimentare la tensiunea specificată pentru alimentarea uC-ului și a celorlalte componente.

Unitate de Gestionare a Energiei: Aceasta este o componentă suplimentară care ajută la gestionarea eficientă a energiei. Poate include circuite pentru comutarea între modurile de alimentare, controlul alimentării senzorului și gestionarea modului de somn al uC-ului.

Microcontroller 328P: Este microcontrollerul central, care controlează întregul sistem. Acesta poate fi programat să controleze alimentarea senzorului, să gestioneze comunicarea și să stocheze datele în memoria internă sau într-o memorie externă.

Senzor: Este componenta care măsoară și furnizează datele necesare. Alimentarea senzorului poate fi controlată de uC, astfel încât să fie activată numai când sunt necesare datele.

Memorie de Stocare: Aceasta poate fi o memorie internă a uC-ului sau o memorie externă (de exemplu, EEPROM sau card SD), care stochează datele măsurate de senzor pentru a fi accesate ulterior.

Exemplu problema rezolvata:

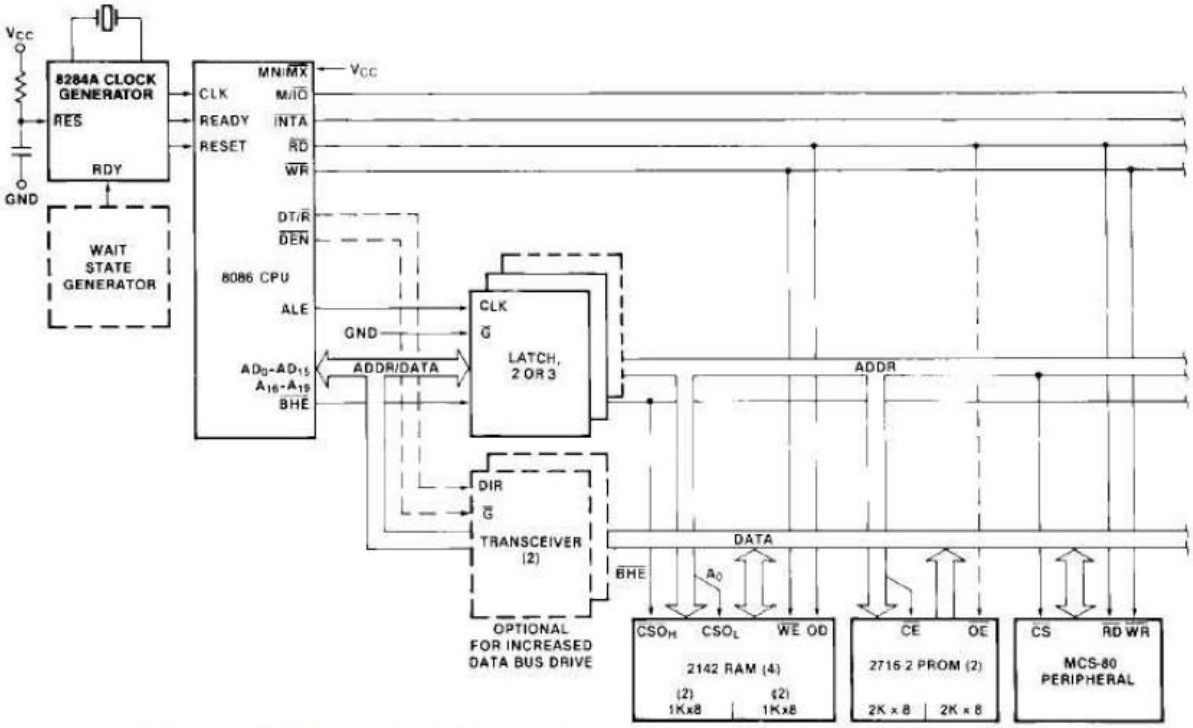
Sa se proiecteze o structura cu 8086 ce contine:
· memorie EPROM de 64k mapata in spatiul 1M-64k ®1M (cipuri 8K * 8)
· sistem de intreruperi
· timer cu ceas de 5MHz ; generaza intreruperi la fiecare 20ms, avind adresa de baza 0x20h.
Sistemul este prevazut cu un circuit de urmarire a acceselor de scriere la portul 0x70h.
Toate datelel scrise la acest port sint stocate intr-o memorie SRAM de 2k * 8 (un singur cip), mapata in spatiul 512k®514k. Cind memoria este umpluta se genereaza o intrerupere pe IRQ3. Memoria SRAM este accesata de procesor numai pe 8 biti.
Se va folosi un numarator binar (pe 11biti) pentru contorizarea acceselor la portul 0x70h.
Se cer:
1) Schema bloc, cu evidentierea modului de functionare a procesorului (mod maxim/minim).
2) Detalierea memorie EPROM (schema , mapare).
3) Detalierea timerului (schema , initializarea modului de lucru, setarea divizorului)
4) Schema bloc a circuitului de urmarire a acceselor la portul 0x70h

Rezolvare

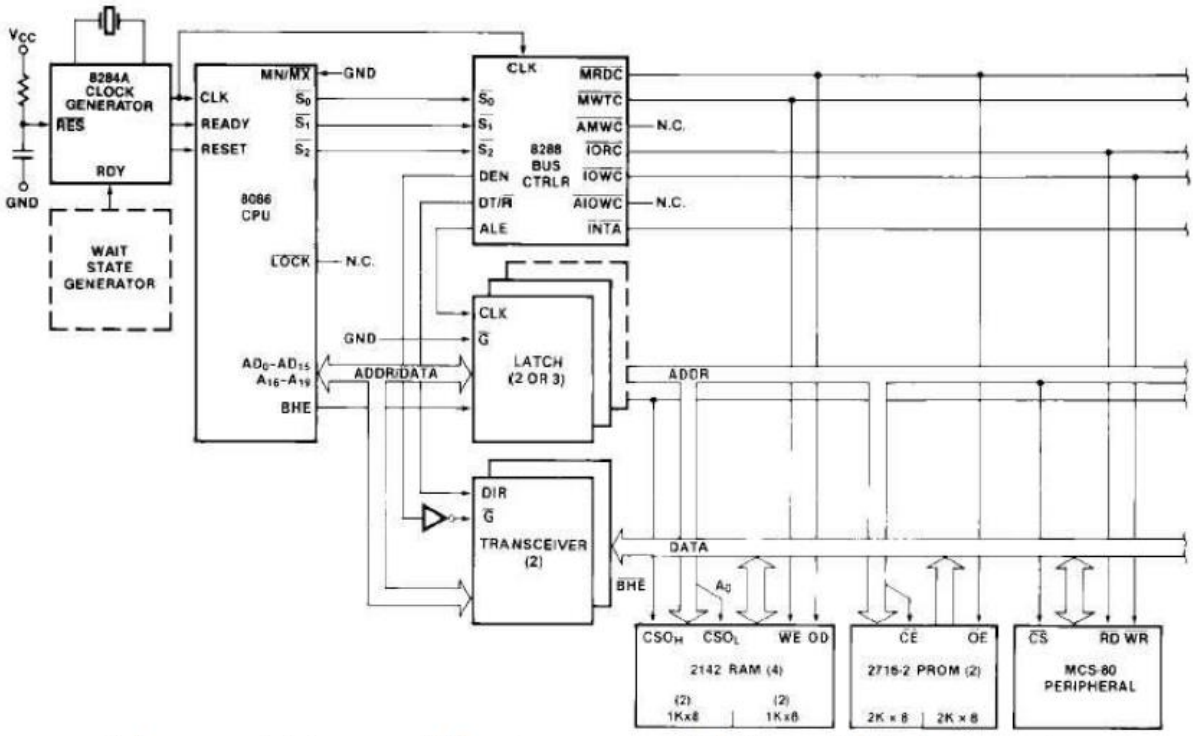
1) In alcatuirea schemei intra urmatoarele blocuri:
Procesorul 8086 – trebuie stabilit de la inceput modul de lucru al procesorului. In cazul de fata, diferentele dintre cele doua moduri de lucru sunt importante prin prisma semnalelor pe care le ofera CPU. De asemenea, daca procesorul este configurat in mod de lucru **maxim**, in schema intervine un bloc in plus, si anume **controllerul de magistrala (bus controller).**
Memoria EPROM – in schema bloc nu se cer detalii de implementare pentru aceasta memorie, ci doar modul in care ea este conectata la magistralele sistemului.
Sistemul de intreruperi – acesta este implementat cu ajutorul circuitului 8259, care trebuie conectat la magistrale.
Timer-ul – pentru implementarea acestuia se foloseste un circuit 8254.
Circuit de urmarire accese la port – pentru conectarea blocului in sistem trebuie identificate intrarile si iesirile acestuia (blocul include si memoria SRAM de inregistrare a acceselor). Astfel, conexiunile vor fi cu :
- magistrala de adrese – blocul **primeste** adrese (conexiune read-only)
- magistrala de date – blocul **citeste/scrie** date (conexiune bidirectionala)
- semnalele de citire/scriere in memorie/spatiul IO (atentie! acestea difera functie de modul de lucru ales al procesorului).

- sistemul de intreruperi – blocul genereaza semnal de intrerupere

Pentru conectarea tuturor acestor blocuri, se poate porni de la schemele din curs sau de la cele oferite de catalogul Intel. Acestea din urma sunt prezentate in continuare, in ambele variante (mod minim si mod maxim).



Schema unui sistem care foloseste procesorul 8086 in modul de lucru **minim**.



Schema unui sistem care foloseste procesorul 8086 in modul de lucru **maxim**.

2) Detalierea memoriei EPROM (schema, mapare)
Se cere proiectarea si maparea unei memorii EPROM in ultimii 64Ko ai spatiului de adresare al procesorului, folosindu-se cipuri de 8K x 8.
Care sunt pasii necesari pentru aceasta operatie?
a. Identificarea numarului de circuite necesare pentru implementarea memoriei.
Pentu cazul de fata avem nevoie de 8 cipuri de felul celor specificate in problema.
b. Stabilirea modului de adresare pentru cipuri.
Trebuie pornit de la semnificatia datelor pentru cip.
Retineti : primul numar (in cazul de fata 16K) implica numarul de pini de intrare (adresa), iar cel de-al doilea reprezinta numarul de biti/semnale de iesire (date).
O memorie de 16k x 8 este o memorie cu 16 * 1024 cuvinte de date de cate 8 biti.
Aceasta inseamna ca la fiecare adresa se afla un cuvant de 8 biti (sau, altfel spus, doua adrese consecutive difera printr-un cuvant de date, nu printr-un bit).
16k adrese => 2_4 * 2_{10} adrese => 10 + 4=14 biti de adresa.
Asadar, din adresa de 20 de biti pe care o furnizeaza procesorul, 14 vor fi folositi pentru a identifica un cuvant in cadrul unui cip EPROM.

Cateva exemple de celule:

Tip cip	**Numar biti/pini adresa**	**Pentru accesare date**			**Capacitate in octeti**
		pe 8 biti	**pe 16 biti**	**pe 32 biti**	
8K x 8	13 (2^3 * 2^{10} adrese)	1 celula	2 celule in paralel	4 celule in paralel	8K
16K x 8	14 (2^4 * 2^{10} adrese)	1 celula	2 celule in paralel	4 celule in paralel	16K
32K x 8	15 (2^5 * 2^{10} adrese)	1 celula	2 celule in paralel	4 celule in paralel	32K
8K x 16	13 (2^3 * 2^{10} adrese)	-	1 celula	2 celule in paralel	16K

Atentie! Verificati enuntul problemei pentru a vedea in ce fel trebuie accesate datele din memoria EPROM. In cazul structurii cu 8086, datele trebuie sa fie accesibile si pe 8 si pe 16 biti. Automat, celulele vor fi grupate in perechi.
Fiecare cip are doua semnale importante:
- CS (semnal negat) – chip select – numai daca pe pinul respectiv se trimite valoarea 0, circuitul este activ.
- OE (semnal negat) – output enable – daca pe acest pin se trimte 0, circuitul trimite datele la iesire.
Avem deci 4 perechi de celule de memorie. La un moment dat, numai un cip sau cel mult o pereche de cipuri poate fi activa (pentru citire pe 8, respectiv 16 biti).

c. Adresarea celulelor
Pentru a citi din memoria ROM, procesorul trimte o adresa de 20 de biti. Datele cerute se

afla numai in unul dintre cele 8 cipuri (in cazul accesului pe 8 biti) sau in una dintre perechi (in cazul accesului pe 16 biti). Se pune problema decodificarii respectivei adrese astfel incat sa fie accesata corect locatia de memorie. Retineti ca procesorul NU stie in ce fel este organizata memoria (in cate cipuri si de ce fel sunt acestea) si nici nu este influentat in nici un fel de aceasta asezare.
Care sunt pasii de decodificare ?

c1. EPROMSEL

Se defineste un semnal, EPROMSEL, care este „responsabil" de selectarea spatiului memoriei EPROM. Definirea acestui semnal este echivalenta cu maparea memoriei in spatiul precizat. In cazul acestei probleme, memoria trebuie mapata in ultimii 64KB ai spatiului de adrese, care are 1MB.
Ultima adresa din EPROM :
1111 1111 1111 1111 1111.
Prima adresa din EPROM :
1111 0000 0000 0000 0000
(64K = 2_6 * 2_{10} => 6+10 = 16 biti 0)
Se observa ca toate adresele din EPROM incep cu 4 biti 1. Altfel spus, memoria EPROM este selectata de orice adresa in care bitii $A_{20}A_{19}A_{18}A_{17}$ sunt 1.
Deci, EPROMSEL = A_{19} Ù A_{18} Ù A_{17} Ù A_{16}

c2. CS0 – CS3

Aceste semnale sunt semnalele „chip select" pentru fiecare linie (pereche) in parte. Pentru ca se cere ca ambele cipuri de pe o linie sa poata fi accesate simultan, semnalul CS va fi comun.

Pentru alegerea uneia dintre cele 4 linii se vor folosi bitii A_{15}-A_{14}.

A15	A14	Semnal
0	0	CS0 – selectare linia 0
0	1	CS1 – selectare linia 1
1	0	CS2 – selectare linia 2
1	1	CS3 – selectare linia 3

Pentru obtinerea acestor semnale in acest fel, solutia cea mai simpla este folosirea unui decodificator. Acesta va fi activat (prin semnalul ENABLE de care dispune) chiar de EPROMSEL, astfel incat o linie este activata numai daca adresa este in spatiul EPROM-ului.

c3. Adresa in cip.

In cadrul fiecarui cip, adresarea se face pe 13 biti. Acestia sunt bitii A_{13}-A_1, care sunt furnizati ca intrari de adresa pentru fiecare cip in parte. Citirea are loc numai pentru cipurile in care CS este activ.

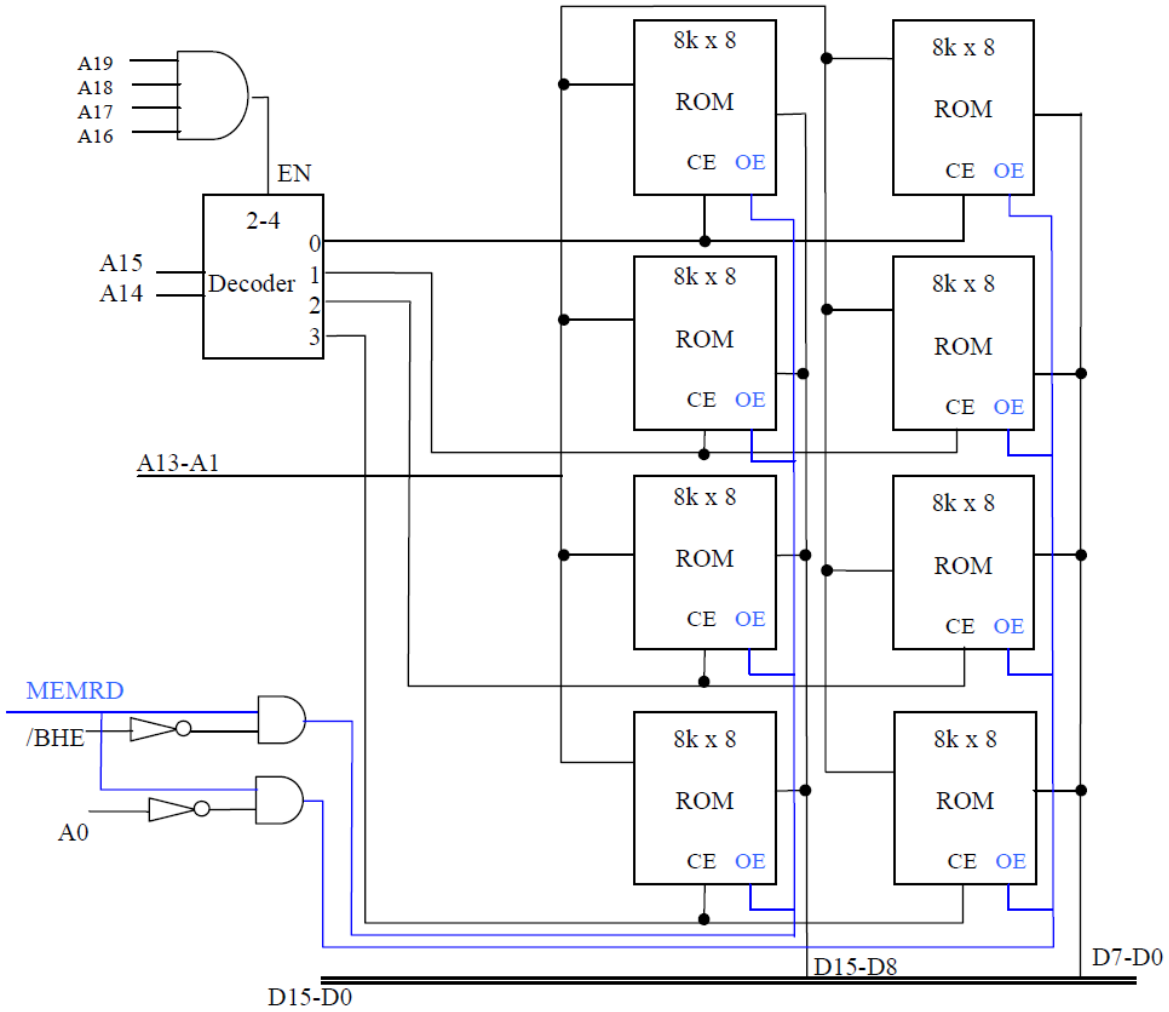
c4. Date disponibile pe 8/16 biti

Datele din EPROM sunt organizate in octeti: adresele pare intr-o coloana (cea din dreapta), cele impare in cealalta coloana (cea din stanga).
Daca se doreste citirea datelor pe 8 biti, in linia selectata cu CS se va alege numai una dintre

celule. Pentru a determina ca una singura dintre celule sa ofere date se foloseste semnalul OE, care va fi activ pentru celula adresata, si inactiv pentru perechea acesteia. Aceasta alegere se face in functie de bitii A0 si BHE. Datele citite sunt puse pe magistrala de date pe liniile corespunzatoare: fie D_0-D_7 pentru adresele pare, fie D_8-D_{15} pentru adresele impare.
Modul de lucru al semnalelor respective este prezentat tabelul urmator. Retineti ca aceasta varianta este o restrictie obligatorie pentru procesorul 8086. (O arhitectura „proprie" poate sa foloseasca doar bitul A0 pentru selectarea octetului de citit).

/BHE	A0	Date
0	0	Date citite pe 16 biti, de la adrese pare, pe D_0-D_{15}
0	1	-
1	0	Date citite pe 8 biti, de pe D_0-D_7
1	1	Date citite pe 8 biti., de pe D_8-D_{15}

In concluzie, schema memoriei EPROM este prezentata in continuare:



Semnalele care sunt transmise pe pinii OE ai cipurilor de memorie se numesc EPROMLBE si EPROMHBE (EPROM Low Byte Enable , respectiv EPROM High Byte Enable).

__
EPROMLBE = MEMRD Ù A0
EPROMHBE = MEMRD Ù BHE
O alta varianta pentru rezolvare – in cazul in care nu se folosesc semnalele OE – este introducerea unor buffere 3-state pentru conectarea celor doi octeti la magistrala. In acest fel, semnalele EPROMLBE si EPROMHBE sunt folosite ca semnale de enable pentru cele 2 buffere. Expresiile lor trebuie putin modificate pentru aceasta varianta:

___
EPROMLBE = EPROMSEL Ù A0
EPROMHBE = EPROMSEL Ù BHE
In concluzie, rezolvarea acestui punct presupune obtinerea unei scheme echivalenta cu cea de mai sus si evidentierea modului in care sunt folositi bitii de adresa pentru a obtine rezultate corecte.
Atentie! O parte dintre semnale pot fi obtinute in moduri diferite in functie de modul de lucru al procesorului.

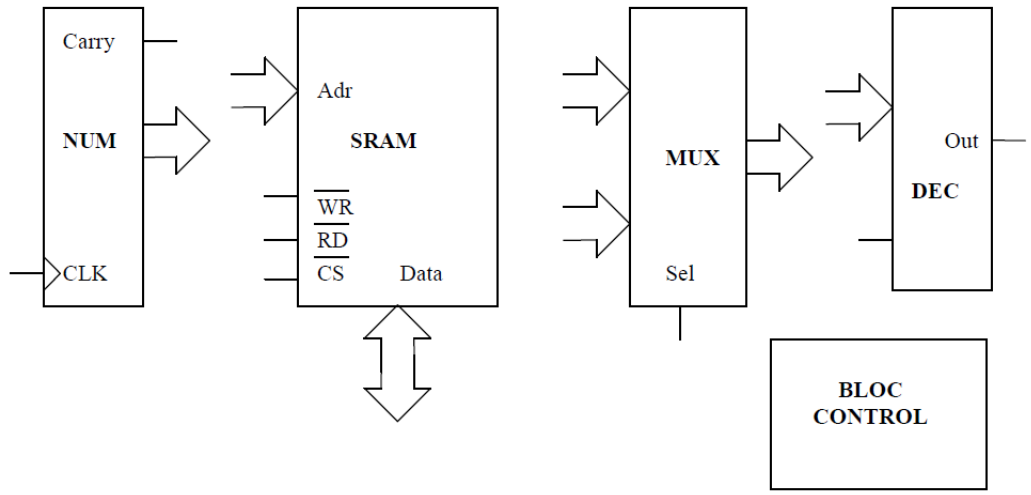
3) Detalierea timer-ului.
Dupa cum se stie, pentru a implementa un astfel de timer se va folosi circuitul 8253.
Rezolvarea are 2 pasi:
- se introduce circuitul in schema, in functie de adresa la care este el mapat
- se configureaza circuitul astfel incat sa se obtina intreruperile la intervalele dorite
Pentru prima parte, rezolvarea este asemanatoare cu aceea din punctul anterior. Mai exact, timer-ul are un semnal CS (semnal negat) care trebuie sa fie activat (deci egal cu 0) in cazul in care adresa de pe magistrala de adrese este 0x20.
Solutia cea mai simpla ar fi maparea „directa”, in sensul ca se poate scrie functia logica prin intermediul careia sa se obtina TIMERCS = 0 in cazul in care bitii de pe magistrala de adresa au configuratia : 0000 0000 0000 0010 0000. (in ordinea A19-A0).
O astfel de logica functioneaza, dar este foarte costisitoare si deloc scalabila. In acest caz, se foloseste de fapt un decodificator pentru spatiul de adrese I/O care sa permita conectarea mai multor astfel de periferice. Avand in vedere ca in general adresele pentru periferice se pun la porturile de forma 0xN0, decodificatorul primeste la intrare bitii A8,A7,A6,A5 si returneaza semnale de tip CS pentru 16 dispozitive, aflate la adresele intre 0x00 si 0xF0. Pentru ca maparea sa fie corecta, decodificatorul primeste ca semnal de ENABLE o functie de genul :
$EN = (IORD + IOWR) \times A_{20} \times A_{19} \times ... \times A_9$

Se pot genera multe alte variante. Important este ca semnalele CS sa asigure ca un singur dispozitiv I/O este activ la un moment dat.
Pentru conectarea la magistrale, mai trebuie facute doar cateva precizari:
Timer.RD = IORD
Timer.WR = IOWR
Timer.CS = TIMERCS (calculat/decodificat anterior)
Timer.A1 = Address.A1
Timer.A0 = Address.A0

Partea a doua este cea care tine de functionalitatea timerului.
Pentru a putea obtine dintr-o freventa de 5MHz un tact cu frecventa de 20ms, trebuie o divizare cu 100000.
De ce?
Daca timerul numara pe frecventa proprie, el furnizeaza intreruperi la :
$1/ (5 * 10^6) = 2 * 10^{-7}$ secunde.
Se cere o marire a intervalului la $2*10^{-2}$ secunde.
Rezulta deci ca este nevoie de divizarea cu 10^5, adica 100000.
Se va folosi modul 0 („interrupt on terminal count”), ceea ce inseamna ca numara de la valoarea setata pana la 0, si apoi genereaza o intrerupere.
Datorita faptului ca numaratoarele din timer sunt de 16 biti, numarul maxim care poate fi setat in cadrul unui numarator este $2^{16} = 64K < 100000$. Vor trebui deci folosite doua dintre numaratoarele lui 8253. Ambele vor fi folosite in modul 0. Primul va numara de la 50000 dupa ceasul propriu (urmand sa dea cate o intrerupere la fiecare 10ms). Cel de-al doilea va primi pe intrarea de ceas iesirea primului, si va numara de la 2, ceea ce inseamna ca va furniza intrerupere la 20ms.
Lista conexiunilor este urmatoarea:
Timer.CLK0 = CLK_5MHz
Timer.CLK1 = Timer.Out0
Secventa de initializare trebuie sa tina cont de :
- adresa portului de control (A1=1, A0=1 => 0x23)
- ambele numaratoare (counter0 si counter1) trebuie programate, fiecare la adresa lui (0x20 si 0x21).
Rezolvarea trebuie sa contina si schema 8253 (mai ales pentru a se evidentia modul in care sunt conectate semnalele) si „programul” de intializare (cuvintele de intializare si porturile la care acestea trebuie scrise).

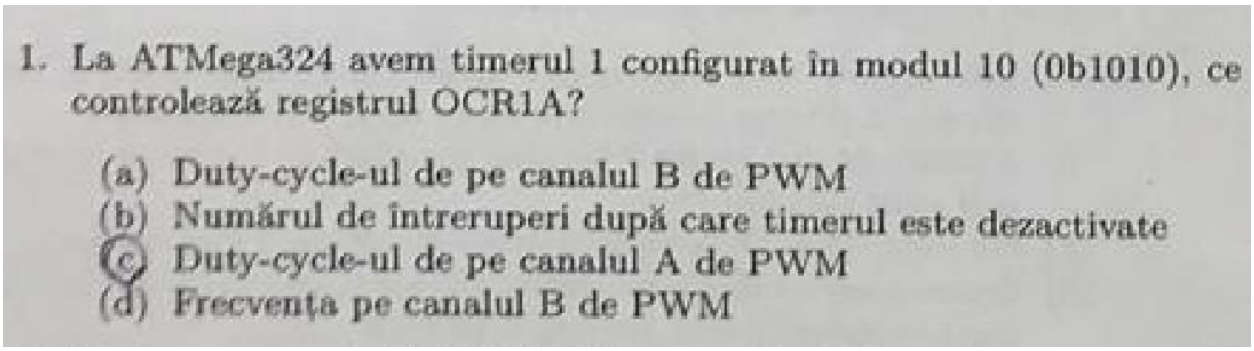
4) Circuitul de supraveghere a acceselor la portul 0x70
Schema bloc a acestui circuit se bazeaza pe urmatoarele blocuri:
- Decodificator adresa port (DEC). Acesta primeste ca intrare adresa de pe magistrala de adrese supravegheata si activeaza iesirea daca a primit adresa corespunzatoare perifericului care trebuie supravegheat
- Un numarator pe 11 biti (NUM), care va genera urmatoarea adresa din memoria SRAM in care se scriu datele. Are ca intrare semnalul de ceas (si eventual un semnal de reset, pentru a fi adus la 0).
- Blocul de memorie (SRAM). Acesta este accesibil pentru scriere (semnalul WR este activat de o scriere la portul 0x70) sau pentru citire (semnalul RD este activat chiar de procesor).Semnalul CS se obtine din adresa la care este mapat blocul de memorie (respectiv 512K).
- Un multiplexor (MUX) care va selecta – in functie de actiunea asupra memoriei – adresa care trebuie furnizata. Astfel, daca are loc o operatie de citire, va fi data adresa de la procesor (mai exact ultimii 11 biti), iar daca se incearca o scriere adresa este funrnizata de numarator.

- Un bloc de control, care asigura (eventual) interfata cu semnalele pe care le furnizeaza procesorul si cu sistemul de intreruperi.



Grile vechi:

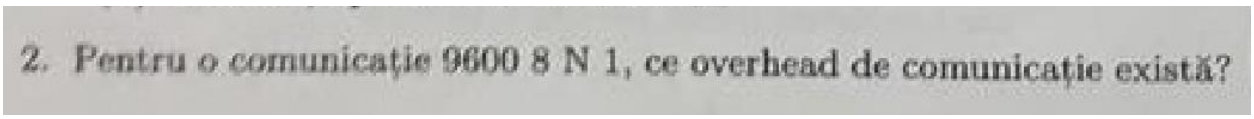
0b1010 ocr1A timer modul 10



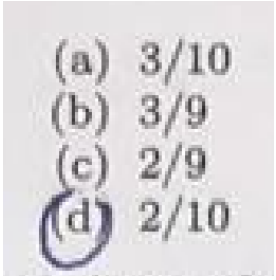
1. La ATMega324 avem timerul 1 configurat în modul 10 (0b1010), ce controlează registrul OCR1A?

   (a) Duty-cycle-ul de pe canalul B de PWM
   (b) Numărul de întreruperi după care timerul este dezactivate
   (c) Duty-cycle-ul de pe canalul A de PWM
   (d) Frecvența pe canalul B de PWM

9600 8 N 1 overhead



2. Pentru o comunicație 9600 8 N 1, ce overhead de comunicație există?



(a) 3/10
(b) 3/9
(c) 2/9
(d) 2/10

Tratare intreruperi AVR memoria de program

3. Unde se află codul pentru rutinele de tratare ale întreruperilor la AVR?

(a) În memoria de program
(b) În tabela de întreruperi
(c) În memoria perifericului de întreruperi
(d) În memoria de date

Romsel memoria mapata adresa 0Xf0000

Ce valoare ar trebui să ia ROMSEL pentru ca memoria din figură să fie mapată începând cu adresa 0xF0000?

(a) $ROMSEL = \overline{A_{19}} A_{18} \overline{A_{17}} A_{16} \overline{A_{15}}$
(b) $ROMSEL = \overline{A_{19}} A_{18} \overline{A_{17}} \overline{A_{16}} \overline{A_{15}}$
(c) $ROMSEL = A_{19} A_{18} A_{17} \overline{A_{16}} A_{15}$
(d) $ROMSEL = A_{19} A_{18} A_{17} A_{16} A_{15}$

Adc AtMega324 ADC

5. O valoare pe ADC la ATMega324 este 100 pentru o referință de 5V. Dacă am fi pus referința la 2.56V, cât ar fi fost valoarea ADC-ului?

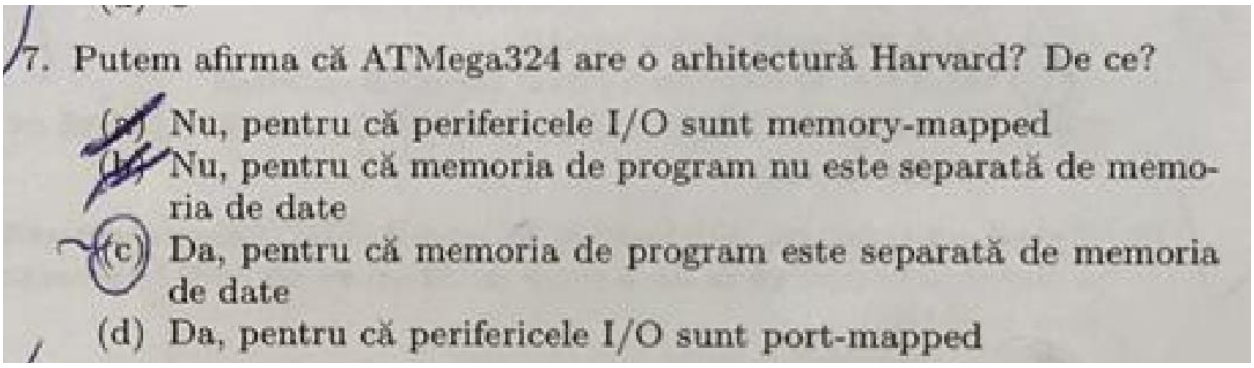
(a) 195
(b) 136
(c) 1023
(d) 0

Conversi ADC Atmega324 simultan

Câte conversii se pot face simultan pe ADC-ul de la ATMega324?

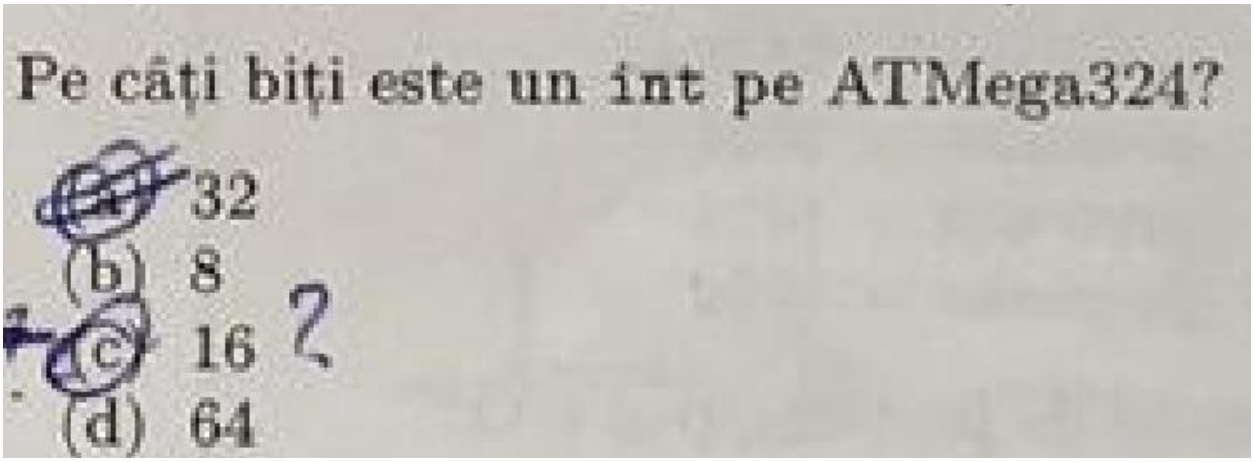
(a) 4
(b) 1
(c) 2
(d) 8

Atmega324 arhitectura harvard


7. Putem afirma că ATMega324 are o arhitectură Harvard? De ce?

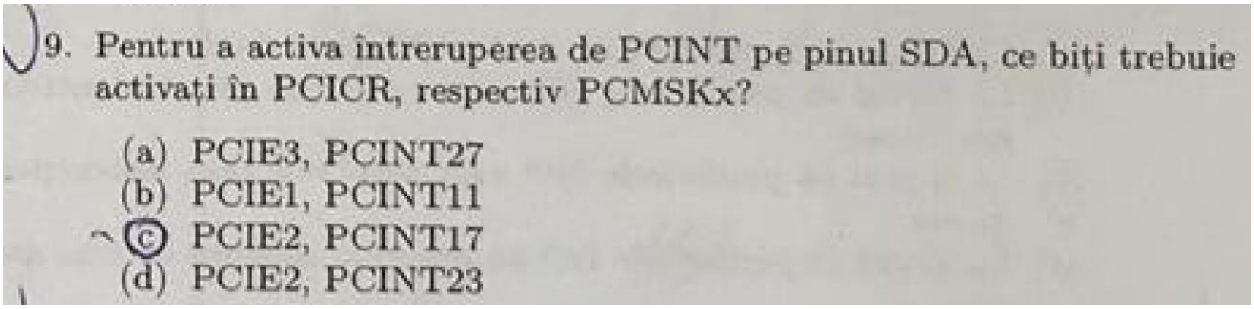
(a) Nu, pentru că perifericele I/O sunt memory-mapped
(b) Nu, pentru că memoria de program nu este separată de memoria de date
(c) Da, pentru că memoria de program este separată de memoria de date
(d) Da, pentru că perifericele I/O sunt port-mapped

Biti int Atmega324


Pe câți biți este un `int` pe ATMega324?

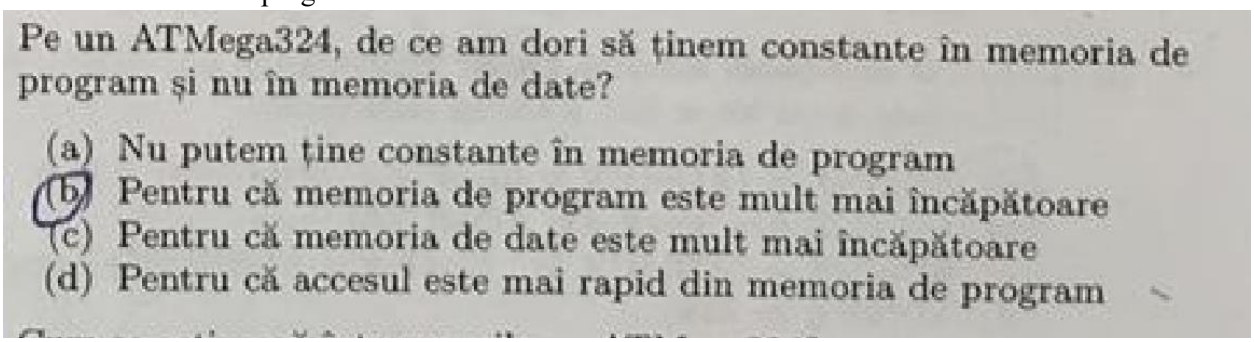
(a) 32
(b) 8
(c) 16
(d) 64

Pcint pin sda biti pcicr pcmskx


9. Pentru a activa întreruperea de PCINT pe pinul SDA, ce biți trebuie activați în PCICR, respectiv PCMSKx?

(a) PCIE3, PCINT27
(b) PCIE1, PCINT11
(c) PCIE2, PCINT17
(d) PCIE2, PCINT23

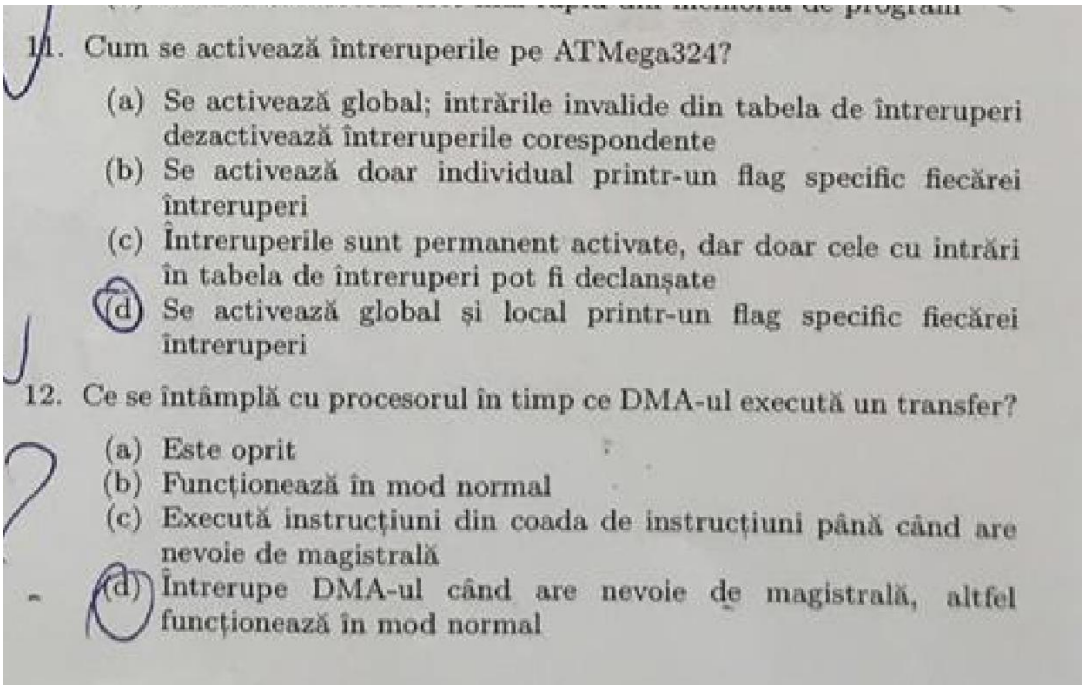
Constant memoria de program memorie de date


Pe un ATMega324, de ce am dori să ținem constante în memoria de program și nu în memoria de date?

(a) Nu putem ține constante în memoria de program
(b) Pentru că memoria de program este mult mai încăpătoare
(c) Pentru că memoria de date este mult mai încăpătoare
(d) Pentru că accesul este mai rapid din memoria de program

Intreruperi atmega324

Procesor dma transfer



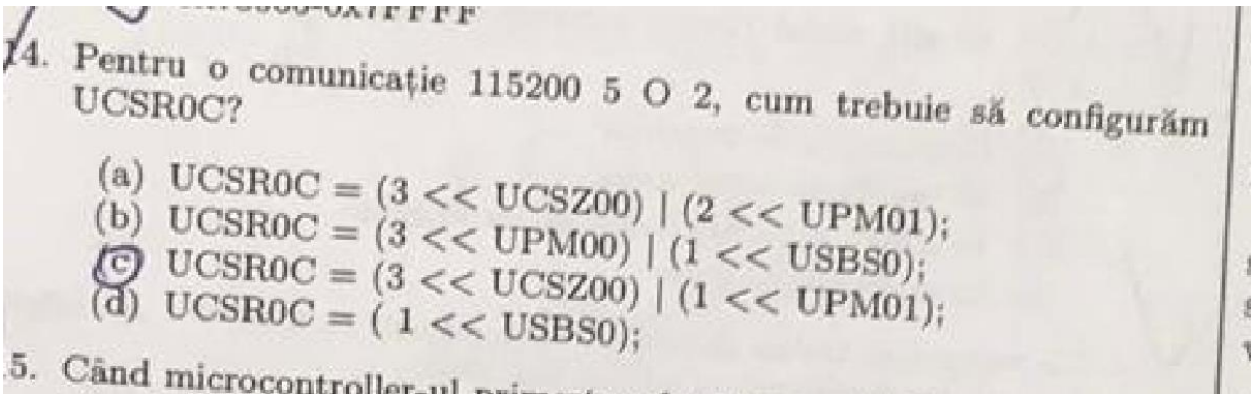
11. Cum se activează întreruperile pe ATMega324?

(a) Se activează global; intrările invalide din tabela de întreruperi dezactivează întreruperile corespondente
(b) Se activează doar individual printr-un flag specific fiecărei întreruperi
(c) Întreruperile sunt permanent activate, dar doar cele cu intrări în tabela de întreruperi pot fi declanșate
(d) Se activează global și local printr-un flag specific fiecărei întreruperi

12. Ce se întâmplă cu procesorul în timp ce DMA-ul execută un transfer?

(a) Este oprit
(b) Funcționează în mod normal
(c) Execută instrucțiuni din coada de instrucțiuni până când are nevoie de magistrală
(d) Întrerupe DMA-ul când are nevoie de magistrală, altfel funcționează în mod normal

Comuncatie ucsr0c



14. Pentru o comunicație 115200 5 O 2, cum trebuie să configurăm UCSR0C?

(a) UCSR0C = (3 << UCSZ00) | (2 << UPM01);
(b) UCSR0C = (3 << UPM00) | (1 << USBS0);
(c) UCSR0C = (3 << UCSZ00) | (1 << UPM01);
(d) UCSR0C = (1 << USBS0);

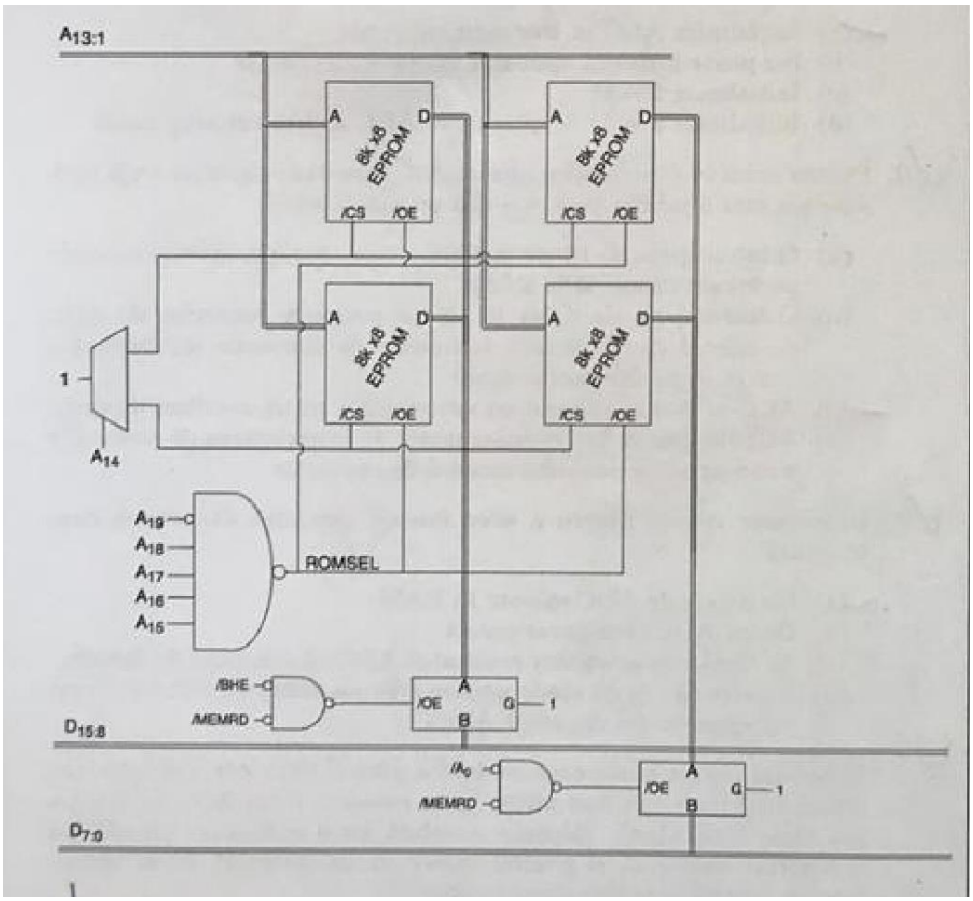
15. Când microcontroller-ul

Spatiu de memorie mapata memoria

13. În ce spațiu de memorie este mapată memoria din figură?

(a) 0x08000-0x0FFFF
(b) 0x08000-0x17000
(c) 0x78000-0x87FFF
(d) 0x78000-0x7FFFF

14. Pentru



Vreți să construiți un roboțel de tip line-follower cu 5 senzori optici analogici. Motoarele sunt comandate cu câte o punte H. Motoarele sunt montate pe fiecare roată mare a robotului (2). Roboțelul este alimentat printr-o baterie la 5V și are baterii separate de 12V pentru motoare.

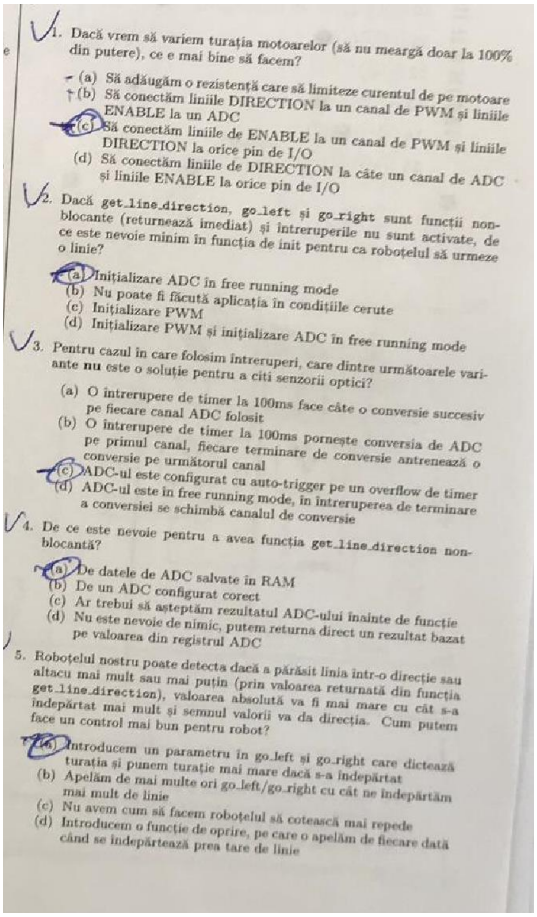
Pentru fiecare dintre cei 5 senzori optici: * Dacă tensiunea este sub 2.5V, sub senzor este alb. Atlfel, este negru/intunecat.

Pentru driverele de motoare, fiecare motor este controlat prin: * un semnal digital de DIRECTION (0 motorul se duce în spate, 1 în față) * un semnal digital de ENABLE (0 motorul este lăsat liber, 1 motorul merge la viteză maximă)

```
int main() {
  init();

  while(1) {
    if (get_line_direction() < 0) {
      go_right();
    } else {
      go_left();
    }
  }
```

Line follower



1. Dacă vrem să variem turația motoarelor (să nu meargă doar la 100% din putere), ce e mai bine să facem?
   - (a) Să adăugăm o rezistență care să limiteze curentul de pe motoare
   - (b) Să conectăm liniile DIRECTION la un canal de PWM și liniile ENABLE la un ADC
   - (c) Să conectăm liniile de ENABLE la un canal de PWM și liniile DIRECTION la orice pin de I/O
   - (d) Să conectăm liniile de DIRECTION la câte un canal de ADC și liniile ENABLE la orice pin de I/O

2. Dacă `get_line_direction`, `go_left` și `go_right` sunt funcții non-blocante (returnează imediat) și întreruperile nu sunt activate, de ce este nevoie minim în funcția de init pentru ca robotelul să urmeze o linie?
   - (a) Inițializare ADC în free running mode
   - (b) Nu poate fi făcută aplicația în condițiile cerute
   - (c) Inițializare PWM
   - (d) Inițializare PWM și inițializare ADC în free running mode

3. Pentru cazul în care folosim întreruperi, care dintre următoarele variante **nu** este o soluție pentru a citi senzorii optici?
   - (a) O întrerupere de timer la 100ms face câte o conversie succesiv pe fiecare canal ADC folosit
   - (b) O întrerupere de timer la 100ms pornește conversia de ADC pe primul canal, fiecare terminare de conversie antrenează o conversie pe următorul canal
   - (c) ADC-ul este configurat cu auto-trigger pe un overflow de timer
   - (d) ADC-ul este în free running mode, în întreruperea de terminare a conversiei se schimbă canalul de conversie

4. De ce este nevoie pentru a avea funcția `get_line_direction` non-blocantă?
   - (a) De datele de ADC salvate în RAM
   - (b) De un ADC configurat corect
   - (c) Ar trebui să așteptăm rezultatul ADC-ului înainte de funcție
   - (d) Nu este nevoie de nimic, putem returna direct un rezultat bazat pe valoarea din registrul ADC

5. Robotelul nostru poate detecta dacă a părăsit linia într-o direcție sau altacu mai mult sau mai puțin (prin valoarea returnată din funcția `get_line_direction`), valoarea absolută va fi mai mare cu cât s-a îndepărtat mai mult și semnul valorii va da direcția. Cum putem face un control mai bun pentru robot?
   - (a) Introducem un parametru în go_left și go_right care dictează turația și punem turație mai mare dacă s-a îndepărtat
   - (b) Apelăm de mai multe ori go_left/go_right cu cât ne îndepărtăm mai mult de linie
   - (c) Nu avem cum să facem robotelul să cotească mai repede
   - (d) Introducem o funcție de oprire, pe care o apelăm de fiecare dată când se îndepărtează prea tare de linie

De la un prof 2018:

1. Un timer pe ATMega324 este configurat să dea un semnal PWM de 10Hz. Pe ce pin este semnalul?
a. 4
b. PD5
c. OC2A
d. 20

2. Care dintre următoarele variante de transmisie serială au rata de transmisie a datelor cea mai mare?
a. USART cu baud rate 1MHz când U2X = 0
b. SPI la 1MHz când SPI2X = 1
c. USART cu baud rate 1Mhz când U2X = 1
d. SPI la 1MHz când SPI2X = 0

3. Pentru a măsura valori de la 8 senzori analogici legați direct la ATMega324a, ce trebuie să fac?
a. 8 conversii pe 8 canale
b. 1 conversie pe 8 canale
c. 8 conversii pe 1 canal
d. Nu se pot lega 8 senzori analogici în același timp

Problemă:

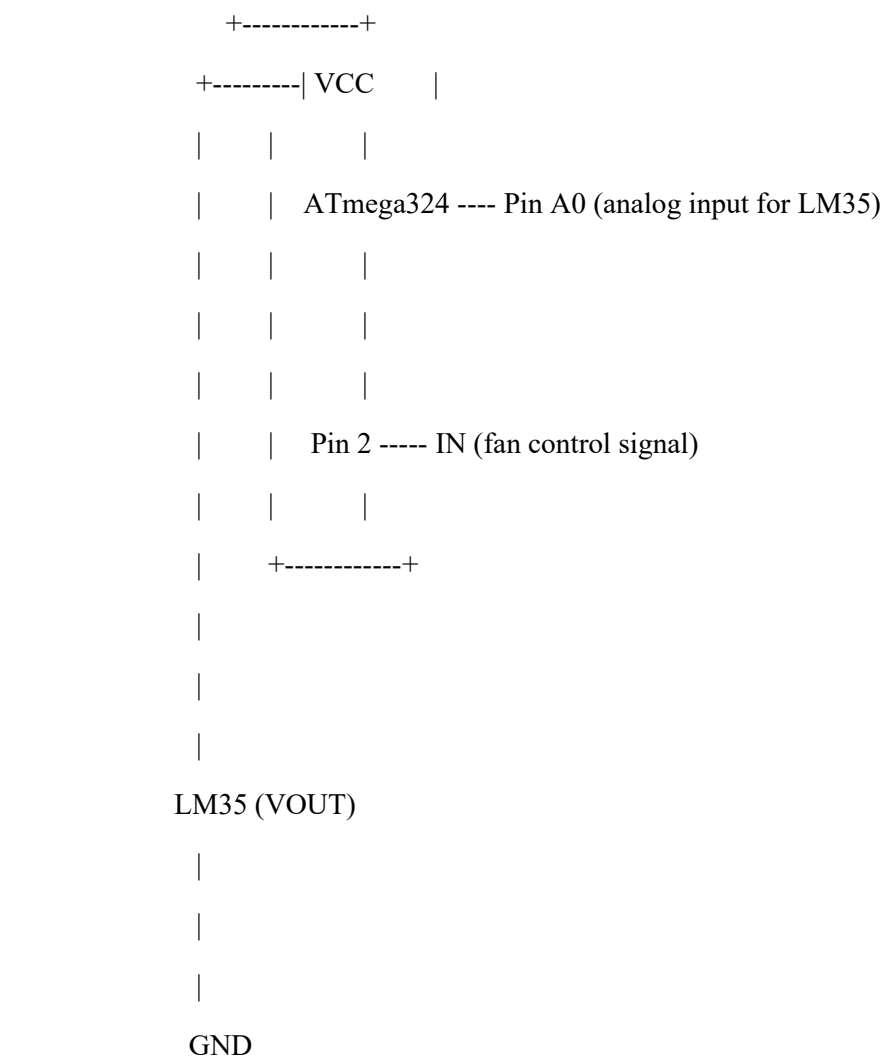
Se cere să se implementeze un dispozitiv de tip "fan controller" cu ajutorul ATMega324. Dispozitivul trebuie să varieze turația unui ventilator în funcție de temperatură.
Aveți la dispoziție:
* Un senzor analogic LM35 cu 3 pini (VCC, GND, VOUT). VOUT va avea o tensiune proporțională cu temperatura, 10mV/°C.
* Un ventilator cu 3 pini (VCC, GND, IN). IN este un semnal de control care va varia turația. Un semnal "1"
constant va da turație maximă, un semnal PWM cu 10% duty cycle va da o turație de 10 ori mai mică.
Trebuie să implementați următoarele funcționalități:
* sub 25°C ventilatorul este oprit
* între 25°C și 40°C ventilatorul are turație între 30% și 100% (variație liniară)
* peste 40°C ventilatorul funcționează la turație maximă
Cerințe:
1 (2p). Descrieți în cuvinte ce periferice I/O veți folosi și care este funcționarea microcontrollerului în aplicație
2 (3p). Faceți schema electrică a aplicației (completă cu asignarea pe pini a diverselor funcții)
3 (5p). Scrieți codul pentru aplicație. Folosiți pe cât posibil întreruperi.

Pentru a implementa dispozitivul "fan controller" cu ajutorul microcontrolerului ATmega324, vom utiliza următoarele periferice I/O:

Un senzor analogic LM35: VCC va fi conectat la o sursă de alimentare de 5V, GND va fi conectat la masă (GND), iar pinul VOUT va fi conectat la un pin analogic (de exemplu, A0) pentru a citi tensiunea proporțională cu temperatura.

Un ventilator: VCC va fi conectat la o sursă de alimentare de 5V, GND va fi conectat la masă (GND), iar pinul IN va fi conectat la un pin digital (de exemplu, pinul 2) pentru a controla semnalul de turație.

Microcontrollerul ATmega324 va fi programat pentru a citi valoarea tensiunii de la senzorul LM35, calcula temperatura corespunzătoare, și în funcție de temperatura, controla pinul IN al ventilatorului pentru a seta turația corespunzătoare.



Cu chatul:

```
#include <Arduino.h>
#define LM35_PIN A0
#define FAN_CONTROL_PIN 2
volatile int temperature = 0;
void setup() {
  analogReference(DEFAULT); // Set analog reference to default (5V)
  pinMode(FAN_CONTROL_PIN, OUTPUT); // Set FAN_CONTROL_PIN as output
  attachInterrupt(digitalPinToInterrupt(LM35_PIN), measureTemperature, CHANGE); // Attach interrupt to LM35_PIN
  sei(); // Enable global interrupts
}
```

```
void loop() {
  // Control turație ventilator
  if (temperature < 25) {
    analogWrite(FAN_CONTROL_PIN, 0); // Oprește ventilatorul
  } else if (temperature >= 25 && temperature <= 40) {
    int speed = map(temperature, 25, 40, 30, 100); // Map temperature to speed range
    analogWrite(FAN_CONTROL_PIN, speed); // Set fan speed
  } else {
    analogWrite(FAN_CONTROL_PIN, 255); // Set maximum fan speed
  }
  delay(100); // Delay for stability
}
void measureTemperature() {
  // Citire valoare analogica de la LM35
  int rawValue = analogRead(LM35_PIN);
  // Calcul temperatura
  float voltage = rawValue * (5.0 / 1023.0); // Convert ADC value to voltage (0-5V)
  temperature = voltage * 100; // Calculate temperature in degrees Celsius
}
```

De pe CC:

Aveti de implementat un sistem de achizitii de date ce va fi alimentat de la o baterie de 3V. Aveti de ales intre unul din urmatoarele modele de uC: 328p si un MSP430FR6972. Ce varianta ati alege? Justificati.

Ce varinata de stabilizator de tensiune ati alege? Justificati.

- Explicati diferenta dintre FastPWM si Phase Correct PWM, explicand modul de implementare al acestora.

- Aveti un sistem hardware de achiztii de date bazat pe un ATmega 328p. Se da un semnal analogic intre (1 - 3V, MAX 5kHz). Propuneti o solutie software pentru a identifica, intr-un mod eficient, frecventa dominanta a semnalului de intrare.

Cu chatul:

Pentru sistemul de achiziții de date alimentat de la o baterie de 3V, având opțiunea de a alege între un microcontroler ATmega328P și un MSP430FR6972, aș alege MSP430FR6972. Motivele pentru această alegere sunt următoarele:

Eficiența energetică: MSP430FR6972 este cunoscut pentru eficiența sa energetică remarcabilă. Are un consum redus de curent și oferă funcționalități pentru gestionarea consumului de energie, cum ar fi modurile de consum extrem de scăzut și capacitatea de a gestiona întreruperi de la senzori în timpul stării de consum scăzut. Aceasta înseamnă că microcontrolerul poate funcționa eficient și mai mult timp pe o baterie de 3V.

Capacități integrate: MSP430FR6972 oferă capacități integrate care sunt utile într-un sistem de achiziții de date. Are un convertor analog-digital (ADC) cu rezoluție de 12 biți, interfețe I2C și SPI și un număr generos de pinii de I/O. Aceste facilități îl fac potrivit pentru achiziționarea și procesarea semnalelor analogice.

Pentru stabilizatorul de tensiune, pentru a menține tensiunea constantă la 3V în ciuda fluctuațiilor de tensiune din bateria de alimentare, aș alege un stabilizator de tensiune cu reglator liniar. Acesta va asigura o tensiune stabilă și va filtra fluctuațiile, ceea ce este important pentru buna funcționare a microcontrolerului și a circuitelor periferice.

Diferența dintre FastPWM și Phase Correct PWM:

FastPWM: Este o tehnică de modulare în lățime a impulsului (PWM) în care durata impulsurilor este fixă, iar frecvența de funcționare poate fi selectată. În modul FastPWM, numărul de cicli de înălțime și scădere a semnalului PWM este maxim, perioada de repetare fiind constantă. Aceasta înseamnă că modulul FastPWM oferă o frecvență de funcționare mai mare și o rezoluție mai mică a lățimii impulsului.

Phase Correct PWM: Este o altă tehnică de modulare în lățime a impulsului (PWM) în care durata impulsurilor variază într-un ciclu complet al semnalului PWM. În modul Phase Correct PWM, semnalul PWM traversează o fază ascendentă și o fază descendentă pentru fiecare ciclu, ceea ce duce la o frecvență de funcționare mai mică, dar o rezoluție mai mare a lățimii impulsului.

Implementarea acestor moduri de PWM în microcontrolere precum ATmega328P implică configurarea registrelor specifice pentru a seta modul corespunzător, prescaler-ul și valorile de comparare.

Pentru a identifica frecvența dominantă a semnalului de intrare (cu valori între 1-3V și o frecvență maximă de 5kHz) într-un sistem hardware de achiziții de date bazat pe microcontrolerul ATmega328P, propun următoarea soluție software:

Configurarea unui pin digital ca intrare pentru a citi semnalul analogic. Vom utiliza ADC-ul (Analog-to-Digital Converter) al microcontrolerului.

Setarea ADC pentru a utiliza referința de tensiune de 5V și o rezoluție de 10 biți.

Configurarea unui timer hardware pentru a genera un semnal de întrerupere (interrupt) la fiecare 1ms.

În funcția de întrerupere, citirea valorii analogice de la pinul de intrare folosind ADC și stocarea acesteia.

Utilizarea unei ferestre de eșantionare pentru a colecta un număr de eșantioane într-un interval de timp prestabilit (de exemplu, 10ms). Putem utiliza un contor pentru a număra eșantioanele și un registru pentru a stoca suma valorilor eșantioanelor.

După colectarea eșantioanelor, calcularea frecvenței dominante utilizând transformata Fourier rapidă (FFT) sau o metodă de analiză spectrală. Există biblioteci și algoritmi disponibili pentru calculul FFT pe platforma Arduino.

Identificarea frecvenței dominante în rezultatele analizei spectrale și utilizarea acestei informații în continuare pentru aplicația specifică.

Este important de menționat că implementarea acestei soluții software poate varia în funcție de cerințele și restricțiile specifice ale sistemului. De asemenea, este posibil să fie necesară calibrarea și ajustarea parametrilor pentru a obține rezultate precise și stabile.

Solutii laburi laborant

Lab 0 ex 2: Folosind butonul conectat la pinul digital 2 (PD2), configurati registri pentru a face pinul de input si faceti LED-ul sa se aprind cand butonul este apasat. (nu uitati sa activati rezistenta de pull-up interna µC-ului)

```
#include <Arduino.h>

// este de preferat sa folosim macrouri pentru a nu hardcoda valori
// daca vrem sa schimbam pinul, acum avem o singura linie de modificat
#define LED PB5
#define BUTTON1 PD2

void setup() {
    // setam pinul LED ca output
    DDRB |= (1 << LED);

    // setam pinul BUTTON1 ca input cu pull-up
    DDRD &= ~(1 << BUTTON1);
    PORTD |= (1 << BUTTON1);
}

void loop() {
    // Citim starea pinului BUTTON1 si aprindem butonul daca este apasat (0 logic
    // deoarece este conectat la GND cand este apasat)
    if ((PIND & (1 << BUTTON1)) == 0) {
        PORTB |= (1 << LED);
    } else {
        PORTB &= ~(1 << LED);
    }
    // Deoarece aceste operatii cu memoria sunt realizate intr-un ciclu de
    // ceas, nu merita sa tinem o variabila de stare
}
```

Lab 0 ex 3: folosind butonul conectat la pinul digital 3 (gasiti voi care este macroul pinului), configurati-l si pe acesta ca sa folositi butoanele pentru a creste si micsora durata delay-ului folosit la exemplul Hello Word.

```
#include <Arduino.h>

#define LED PB5
#define BUTTON1 PD2
#define BUTTON2 PD3

#define TIME_MODIFIER 100
#define LED_TIME 1000
#define MAX_ON_TIME 800
#define MIN_ON_TIME 100

int onTime = LED_TIME - (MAX_ON_TIME + MIN_ON_TIME) / 2;

void setup() {
  // setam pinul LED ca output
  DDRB |= (1 << LED);

  // setam pinul BUTTON1 ca input cu pull-up
  DDRD &= ~(1 << BUTTON1);
  PORTD |= (1 << BUTTON1);

  // setam pinul BUTTON2 ca input cu pull-up
  DDRD &= ~(1 << BUTTON2);
  PORTD |= (1 << BUTTON2);
}

void loop() {
  // citim starea pinului si incrementam on time-ul daca este apasat butonul 1
  if ((PIND & (1 << BUTTON1)) == 0) {
    onTime += TIME_MODIFIER;
    if (onTime > MAX_ON_TIME) {
      onTime = MAX_ON_TIME;
    }
  } else if ((PIND & (1 << BUTTON2)) == 0) { // sau il scadem daca este apasat butonul 2
    onTime -= TIME_MODIFIER;
    if (onTime < MIN_ON_TIME) {
      onTime = MIN_ON_TIME;
    }
  }

  // aprindem led-ul pentru un timp de onTime milisecunde
  PORTB |= (1 << LED);
```

```
  delay(onTime);
  PORTB &= ~(1 << LED);
  delay(LED_TIME - onTime);
}
```

Lab 1 ex 1: Folosind interfața serială și biblioteca Arduino, trimiteți următoarele comenzi dintr-un terminal serial:

“on” - aprinde un led

“off” - stinge un led

“blink” – blink un led la interval de 200 ms

“get” - afișează prin interfața serială starea unui buton

Fiecare comandă va fi urmată de caracterul de control: “\n” (newline). Configurați terminalul serial să trimită (doar) acest caracter după fiecare mesaj. Implementați un program în Arduino care să recunoască aceste comenzi și să realizeze acțiunile corespunzătoare pentru fiecare dintre ele. Folosiți pin-ul 13 (PB5) pentru LED și pin-ul 2 (PD2) pentru buton.

Comanda blink va porni funcția blink (blink led “la infinit”)

Comenzile on/off vor opri funcția blink.

```
void setup()
{
  // pinMode(13, OUTPUT);
  DDRB |= (1 << PB5);
  // pinMode(2, INPUT_PULLUP);
  DDRD &= ~(1 << PD2);
  PORTD |= (1 << PD2);
  Serial.begin(9600);
  delay(100);
  Serial.println("ready");
}

char txbuf[20];
char rxbuf[20];
int index = 0;
long ts = 0;
short blink_on = 0;

void loop()
{
```

```
 if (Serial.available()) {
   char a = Serial.read();
   sprintf(txbuf, "%s: %c", "primit caracter", a);
   Serial.println(txbuf);
   if (a == '\n') {
     rxbuf[index] = '\0';
     Serial.println(rxbuf);
     if (strcmp(rxbuf, "on") == 0) {
       blink_on = 0;
       digitalWrite(13, HIGH);
     }
     if (strcmp(rxbuf, "off") == 0) {
       blink_on = 0;
       digitalWrite(13, LOW);
     }
     if (strcmp(rxbuf, "blink") == 0) {
       blink_on = !blink_on;
       PORTB &= ~(1 << PB5);
     }
     if (strcmp(rxbuf, "get") == 0) {
       Serial.print("button state: ");
       // Serial.println(digitalRead(2));
       Serial.println((PIND & (1 << PD2)) == 0);
     }
     index = 0;
     rxbuf[index] = '\0';
   } else {
     rxbuf[index] = a;
     index++;
     rxbuf[index] = '\0';
   }
 }

 if (blink_on) {
   if ((millis() - ts) >= 100) {
     ts = millis();
     PORTB ^= (1 << PB5);
   }
 }
}
```

Lab 1 ex 2: Rulați exemplul pentru USART pe bază de registre. Pentru configurările serialei, urmăriți fisierul usart.c din schelet.

Configurați USART0 cu următorii parametri: baud rate 19200, 8 biți de date, 1 bit de stop, fara paritate. Transmiteți către PC câte un mesaj pentru fiecare eveniment de apăsare/lăsare a unui buton (ex: se apasă PD2, se transmite “PD2 apăsat”, se lasă PD2, se transmite “PD2 lăsat”, câte o singură dată pe apăsare).

```
#include "usart.h"

void setup()
{
  DDRD &= ~(1 << PD2);
  PORTD |= (1 << PD2);

  USART0_init();
  USART0_print("Hello\n");
}

short state = 0;
short count = 0;
char buf[20];

void loop()
{
  switch (state) {
    case 0:
      if ((PIND & (1 << PD2)) == 0) {
        sprintf(buf, "%s: %d\n", "PD2 apasat", count);
        count++;
        USART0_print(buf);
        state = 1;
      }
      break;
    case 1:
      if ((PIND & (1 << PD2)) != 0) {
        USART0_print("PD2 lasat\n");
        state = 0;
      }
      break;
  }
}
```

```
#include "usart.h"

/*
  Functie de initializare a controllerului USART
*/
void USART0_init()
```

```
{
  /* seteaza baud rate la 19200 */
  UBRR0 = 51;

  /* porneste transmitatorul */
  UCSR0B = (1 << TXEN0) | (1 << RXEN0);

  /* seteaza formatul frame-ului: 8 biti de date, 1 bit de stop, fara paritate */
  UCSR0C &= ~(1 << USBS0);
  UCSR0C |= (3 << UCSZ00);
}

/*
   Functie ce transmite un caracter prin USART

   @param data - caracterul de transmis
*/
void USART0_transmit(char data)
{
  /* asteapta pana bufferul e gol */
  while (!(UCSR0A & (1 << UDRE0)))
    ;

  /* pune datele in buffer; transmisia va porni automat in urma scrierii */
  UDR0 = data;
}

/*
   Functie ce primeste un caracter prin USART

   @return - caracterul primit
*/
char USART0_receive()
{
  /* asteapta cat timp bufferul e gol */
  while (!(UCSR0A & (1 << RXC0)))
    ;

  /* returneaza datele din buffer */
  return UDR0;
}

/*
   Functie ce transmite un sir de caractere prin USART

   @param data - sirul (terminat cu '\0') de transmis
```

```
*/
void USART0_print(const char *data)
{
  while (*data != '\0')
    USART0_transmit(*data++);
}
```

Lab 1 ex 3: Mesajele trimise de primul Arduino vor fi afisate în terminalul Serial conectat la cel de-al doilea Arduino, și invers.

Aprindeți led-ul (PB5) pentru 500 ms atunci când este recepționat un mesaj. Pentru a nu bloca programul (și implicit recepționarea mesajelor), nu folosiți funcția delay.

Atenție la realizarea conexiunilor RX/TX (RX1 la TX2, TX1 la RX2) și GND (GND1 la GND2, dacă plăcile Arduino sunt conectate la PC-uri diferite)

În caz că firele nu sunt suficient de lungi pentru a conecta direct cele două Arduino (legate în același timp la 2 PC-uri prin cabluri USB), folosiți breadboard-urile și realizați conexiunile prin mai multe fire.

```
#include <SoftwareSerial.h>
SoftwareSerial mySerial(10, 11); // RX, TX

void setup()
{
  DDRB |= (1 << PB5);
  Serial.begin(9600);
  Serial.println("ready");
  mySerial.begin(4800);
  mySerial.println("ready 2");
}

short state = 0;
long timestamp = 0;

void loop()
{
  if (mySerial.available()) {
    Serial.write(mySerial.read());
    PORTB |= (1 << PB5);
    state = 1;
    timestamp = millis();
  }
  if (Serial.available()) {
    mySerial.write(Serial.read());
  }
  if (state == 1) {
```

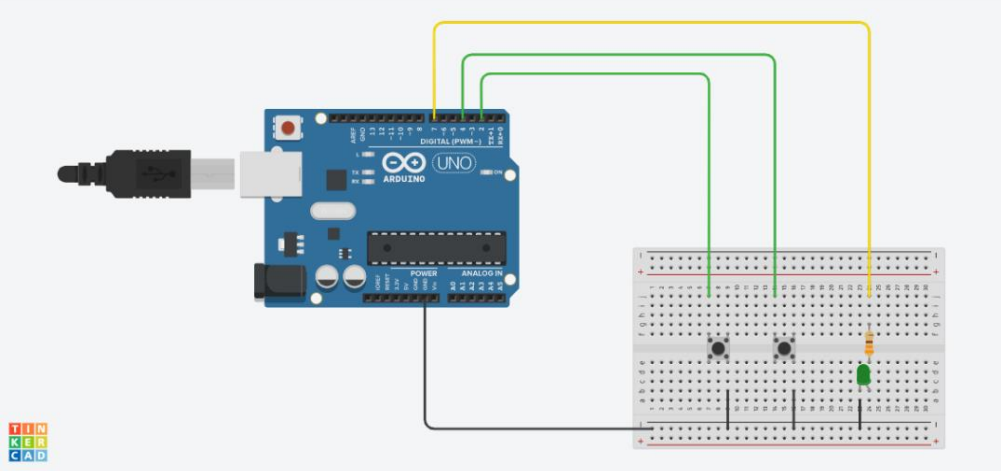
```
  if ((millis() - timestamp) >= 1000) {
    state = 0;
    PORTB &= ~(1 << PB5);
  }
 }
}
```

Lab 2 ex 1: Folosiți întreruperi externe (INT și/sau PCINT) pentru a detecta apăsarea unui buton conectat la PD2 (pin 2) și a unuia conectat la PD4 (pin 4). Modificați starea unui LED conectat la PD7 (pin 7) în ISR.

Configurați întreruperea externă (INT) pe front descrescător (falling edge) sau PCINT-ul corespunzător pinului folosit

Modificați starea LED-ului la apăsarea oricărui buton (PD2, PD4)

Tinkercad - Testați programul folosind următorul montaj:



```
unsigned long t_debounce_1, t_debounce_2 = 0;

ISR(INT0_vect)
{
 // cod întrerupere externă
 unsigned long t_int = millis();
 if (t_int - t_debounce_1 > 300) {
   t_debounce_1 = t_int;
   // test blink
   PORTD ^= (1 << PD7);
 }
}

ISR(PCINT2_vect) {
```

```
  // cod întrerupere de tip pin change
  if ((PIND & (1 << PD4)) == 0) {
    // întrerupere generată de pinul PD4 / PCINT20

    unsigned long t_int = millis();
    if (t_int - t_debounce_2 > 300) {
      t_debounce_2 = t_int;
      // test blink
      PORTD ^= (1 << PD7);
    }
  }
}

void setup_interrupts() {
  // buton 1: PD2 / INT0
  // buton 2: PD4 / PCINT20
  cli();

  // input
  DDRD &= ~(1 << PD2) & ~(1 << PD4);
  // input pullup
  PORTD |= (1 << PD2) | (1 << PD4);

  // configurare intreruperi
  // intreruperi externe
  EICRA |= (1 << ISC01);    // set INT0 to trigger on falling edge (1-0)
  // întreruperi de tip pin change (activare vector de întreruperi)
  PCICR |= (1 << PCIE2); // enable the pin change interrupt, set PCIE2 to enable PCMSK2 scan

  // activare intreruperi
  // intrerupere externa
  EIMSK |= (1 << INT0);     // Turns on INT0
  // întrerupere de tip pin change
  PCMSK2 |= (1 << PCINT20); // Turns on PCINT20 (PD4)

  sei();
}

void setup() {
  setup_interrupts();
  // test led
  DDRD |= (1 << PD7);
  PORTD &= ~(1 << PD7);
}

void loop() {
```

Lab 2 ex 2: Pe baza aceluiași montaj, folosiți întreruperi externe (INT și/sau PCINT) pentru a detecta apăsarea unui buton conectat la PD2 (pin 2) și a unuia conectat la PD4 (pin 4). Modificați intervalul de semnalizare (blink) al unui LED conectat la PD7 (pin 7) în ISR astfel:

butonul conectat la PD2 crește intervalul cu 100 ms

butonul conectat la PD4 scade intervalul cu 100 ms

```
unsigned long t_debounce_1, t_debounce_2 = 0;
int dt = 500;

ISR(INT0_vect) {
  // cod întrerupere externă
  unsigned long t_int = millis();
  if (t_int - t_debounce_1 > 300) {
    t_debounce_1 = t_int;
    dt += 100;
  }
}

ISR(PCINT2_vect) {
  // cod întrerupere de tip pin change
  if ((PIND & (1 << PD4)) == 0) {
    // întrerupere generată de pinul PD4 / PCINT20

    unsigned long t_int = millis();
    if (t_int - t_debounce_2 > 300) {
      t_debounce_2 = t_int;
      if (dt > 100) {
        dt -= 100;
      }
    }
  }
}

void setup_interrupts() {
  // buton 1: PD2 / INT0
  // buton 2: PD4 / PCINT20
  cli();

  // input
  DDRD &= ~(1 << PD2) & ~(1 << PD4);
  // input pullup
  PORTD |= (1 << PD2) | (1 << PD4);

  // configurare intreruperi
  // intreruperi externe
```

```
  EICRA |= (1 << ISC01);    // set INT0 to trigger on falling edge (1-0)
  // întreruperi de tip pin change (activare vector de întreruperi)
  PCICR |= (1 << PCIE2); // enable the pin change interrupt, set PCIE2 to enable PCMSK2 scan

  // activare intreruperi
  // intrerupere externa
  EIMSK |= (1 << INT0);     // Turns on INT0
  // întrerupere de tip pin change
  PCMSK2 |= (1 << PCINT20); // Turns on PCINT20 (PD4)

  sei();
}


void setup() {
  setup_interrupts();
  // test led
  DDRD |= (1 << PD7);
  PORTD &= ~(1 << PD7);
}

void loop() {
  PORTD |= (1 << PD7);
  delay(dt);
  PORTD &= ~(1 << PD7);
  delay(dt);
}
```

Lab 3 ex 0: Folosind Timer 1 de pe Arduino și operațiile cu regiștri, afișați pe interfața serială un mesaj la alegerea voastră o dată la 5 secunde.

Timerul de pe Arduino nu poate cronometra perioade atât de lungi. Veți fi nevoiți să afișați mesajul după un anumit număr de perioade, de lungime aleasă de voi. Ex: perioadă de 0.5s (2Hz), afișăm mesajul la fiecare a doua intrare în funcția asociată întreruperii de timer; perioadă de 0.1s (10Hz), afișăm mesajul la fiecare a 10-a intrare.

Alegerea parametrilor pentru timer pentru setările registrelor:

```
  OCR1A = 31249; // compare match register 16MHz/256/2Hz-1 => 2Hz

  TCCR1B |= (1 << WGM12);   // CTC mode

  TCCR1B |= (1 << CS12);    // 256 prescaler

  TIMSK1 |= (1 << OCIE1A);  // enable timer compare interrupt
```

```
void setup() {
```

```
  cli();

  Serial.begin(9600);

  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1  = 0;

  OCR1A = 31249;            // compare match register 16MHz/256/2Hz-1
  TCCR1B |= (1 << WGM12);   // CTC mode
  TCCR1B |= (1 << CS12);    // 256 prescaler
  TIMSK1 |= (1 << OCIE1A);  // enable timer compare interrupt

  sei();
}

volatile int count = 0;

ISR(TIMER1_COMPA_vect) {
  count++;
  if (count == 11) {
    count = 0;
    Serial.println("Mesaj");
  }
}

void loop()
{

}
```

Lab 3 ex 1: Folosind codul de la exercițiul anterior, la fiecare 5 secunde veți alterna între două stări de redare a unei melodii pe buzzer-ul din montaj:

viteză normală și octavă normală

viteză dublă (noteDuration / 2) și octavă superioară

Creșterea unei octave se realizează dublând frecvența notelor. Pentru redarea unei note veți folosi funcția tone(). Urmăriți exemplele de pe acest repository arduino-songs și alegeți o melodie.

Realizați redarea melodiei în funcția loop() pentru a putea folosi întreruperile generate de timer și pentru a reda continuu melodia.

```
int notes=sizeof(melody)/sizeof(melody[0])/2;

// this calculates the duration of a whole note in ms (60s/tempo)*4 beats
```

```
int wholenote = (60000 * 4) / tempo;

int divider = 0, noteDuration = 0;

void setup() {
  cli();

  TCCR1A = 0;
  TCCR1B = 0;
  TCNT1  = 0;

  OCR1A = 62499;            // compare match register 16MHz/256/1Hz-1
  TCCR1B |= (1 << WGM12);   // CTC mode
  TCCR1B |= (1 << CS12);    // 256 prescaler
  TIMSK1 |= (1 << OCIE1A);  // enable timer compare interrupt
  sei();
}

volatile int melody_select = 0;
volatile int count = 0;

ISR(TIMER1_COMPA_vect) {
  count++;
  if (count == 6) {
    melody_select = 1 - melody_select;
    count = 0;
  }
}

void loop() {
  // iterate over the notes of the melody.
  // Remember, the array is twice the number of notes (notes + durations)
  for (int thisNote = 0; thisNote < notes * 2; thisNote = thisNote + 2) {

    // calculates the duration of each note
    divider = melody[thisNote + 1];
    if (divider > 0) {
      // regular note, just proceed
      noteDuration = (wholenote) / divider;
    } else if (divider < 0) {
      // dotted notes are represented with negative durations!!
      noteDuration = (wholenote) / abs(divider);
      noteDuration *= 1.5; // increases the duration in half for dotted notes
    }

    tone(buzzer, melody[thisNote] *  (melody_select + 1), noteDuration * 0.9 / (melody_select + 1));
```

```
    // we only play the note for 90% of the duration, leaving 10% as a pause

    // Wait for the specief duration before playing the next note.
    delay(noteDuration / (melody_select + 1));

    // stop the waveform generation before the next note.
    noTone(buzzer);
  }
}
```

Lab 3 ex 2: Folosind Timer 0, ciclați led-ul RGB între mai multe culori și mișcați servo-motorul câte 10° la fiecare x secunde (alegeți voi o perioadă potrivită pentru urmărirea schimbărilor). Nu uitați de sugestia de la primul task (va trebui să numărați un număr de perioade mult mai scurte decât cea dorită).

Atenție la limitele de unghi ale servo-motorului. Depășirea acestora poate avaria servomotorul. Verificați ca unghiul să nu depășească intervalul [0°, 180°] (sau [0, pi] pentru cei ce își amintesc cu plăcere de trigonometrie :-P).

```
#include <Servo.h>

Servo myservo;

volatile int step = 10;
volatile int direction = 0;
volatile int pos = 0;

void setup() {
  cli();
  myservo.attach(3);
  pinMode(9, OUTPUT);
  pinMode(10, OUTPUT);
  pinMode(11, OUTPUT);

  TCCR0A = 0;
  TCCR0B = 0;
  TCNT0  = 0;

  OCR0A = 63;           // compare match register 16MHz/256/1000Hz-1
  TCCR0A |= (1 << COM0A0);
  TCCR0A |= (1 << WGM01);
  TCCR0B |= (1 << CS02);    // 256 prescaler
  TIMSK0 |= (1 << OCIE0A);  // enable timer compare interrupt
  sei();

}

volatile int led = 0;
```

```
volatile int count = 0;


ISR(TIMER0_COMPA_vect) {
  count++;
  if (count == 1000) {
    led = 1 - led;
    count = 0;
    if (direction == 0) {
      if (pos <= 180)
      pos += step;
      else
        direction = 1;
    } else if (direction == 1) {
      if (pos >= 0)
        pos -= step;
      else
        direction = 0;
    }
  }
}

void loop() {
  myservo.write(pos);
  if (led){
    analogWrite(9, 0);
    analogWrite(10, 125);
    analogWrite(11, 255);
  }
  else {
    analogWrite(9, 255);
    analogWrite(10, 125);
    analogWrite(11, 0);
  }
}
```